Geavanceerde computerarchitecturen - Session 3

Cas Truyers en Niels Verhoeve

4 maart 2026

1 Introductie

In dit labo onderzoeken we hoe we onze GPU-code efficiënter kunnen maken door rekening te houden met het concept warps, striding en het vermijden van thread divergence. Eerst maken we een kernel die een afbeelding inverteert en passen we striding toe om flexibel om te gaan met de verhouding tussen het aantal threads en de hoeveelheid data. Daarna breiden we dit uit door een 1D-Gaussian blur op de rode pixels toe te passen. Dit doen we om in de praktijk te meten hoe hard thread divergence onze rekentijd vertraagt en hoe we dit kunnen oplossen door onze data structuur aan te passen.

2 Frequency Scaling; Problem and Solution

Door de frequentieschaling van de GPU is er een grote variatie in de performance en dus de execution time van de operatie. Om deze variaties te vermijden, hebben we de kloksnelheid van de GPU met behulp van de nvidia-smi command-line tool vastgezet, wat ons een veel stabieler meetresultaat geeft. Hieronder wordt de graphics clock op 1200 MHz gezet en de memory clock op 5000 MHz, alleen was het op onze GPU niet mogelijk om de memory clock op een vaste frequentie te zetten. Het laatste commando laat toe om de huidige frequenties van de GPU per seconde te monitoren.

```
sudo nvidia-smi -pm 1

nvidia-smi -q -d SUPPORTED_CLOCKS

sudo nvidia-smi -lgc 1200,1200

sudo nvidia-smi -lmc 5000,5000

Every 1.0s: nvidia-smi -q -d CLOCK
```

3 Deel 1: Inverteren en Striding

De GPU groepeert threads in warps van 32 threads. Tot nu toe hebben we één thread toegewezen aan één data-element. Maar als je meer data (pixels) hebt dan beschikbare threads dan implementeren we een grid-stride loop om in plaats van elke thread maar één berekening te laten doen, deze meerdere berekeningen te laten doen. Zo kunnen we werken met een threads-elementen verhouding van bijvoorbeeld 1:16.

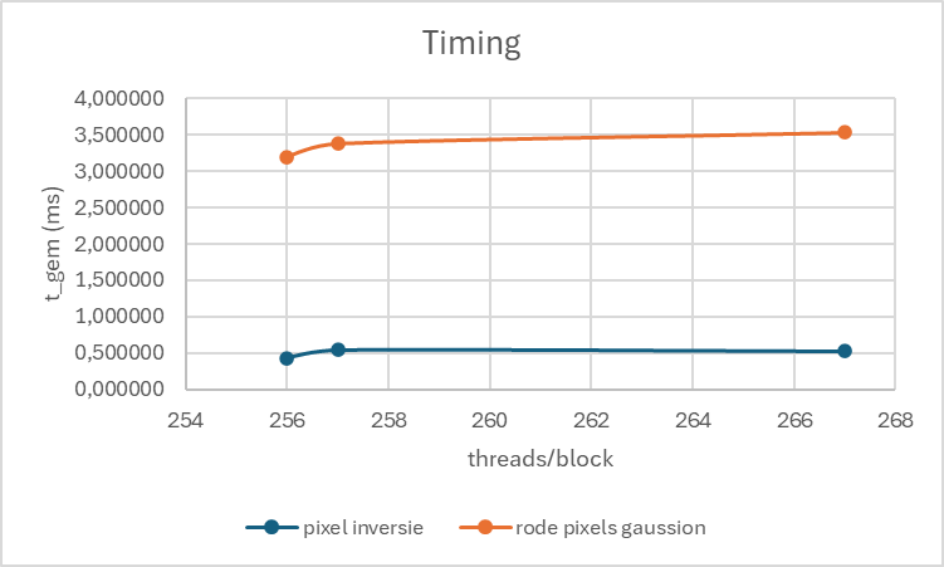
3.1 Invert without Striding

```
__global__ void inverse_Kernel(uint8_t* arrayOut, uint8_t* arrayIn, size_t arraySize)
{
    int i=threadIdx.x+blockIdx.x*blockDim.x;
    if(i<arraySize) arrayOut[i] = 255 - arrayIn[i];
}
```

Uit Tabel 1 is duidelijk dat een blockgrootte van 256 (een veelvoud van 32) het meest efficiënt is omdat we dan geen idle threads hebben. Bij 257 threads per block heeft de GPU voor elk block een extra warp nodig waarvan 31 threads ongebruikt blijven. Dit is het minst efficiënt en heeft de langste uitvoeringstijd. Bij 267 threads is de verhouding tussen actieve (11) en idle (21) threads iets gunstiger, wat de licht verbeterde tijd t.o.v. 257 threads verklaart.

Threads per block	Warps per block	Gemiddelde tijd (ms)
256	8 (volledig)	0.4295
257	8 + 1 (1 actieve thread)	0.5419
267	8 + 1 (11 actieve threads)	0.5245

Tabel 1: Invloed van partial warps (zonder striding).



Figuur 1: tijdsduur bewerkingsoperaties

3.2 Invert with Striding

```
__global__ void inverseWithStriding_Kernel(uint8_t* arrayOut, uint8_t* arrayIn, size_t
    arraySize)
{
    int tid=threadIdx.x+blockIdx.x*blockDim.x;
    int stride = blockDim.x*gridDim.x;

    for(int i = tid; i<arraySize; i+=stride) arrayOut[i] = 255 - arrayIn[i];
}
```

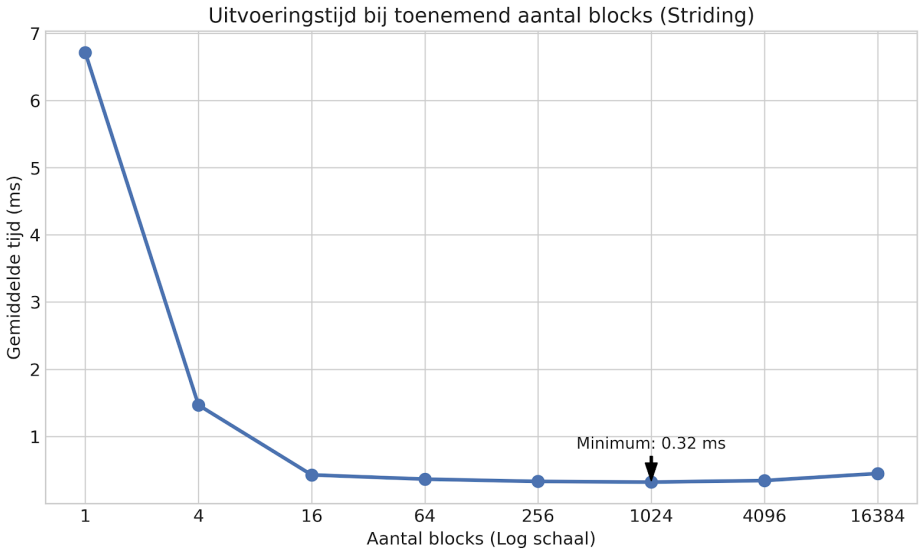
Blocks	Threads/block	Totale threads	Gemiddelde tijd (ms)
1	256	256	6.711
4	256	1024	1.466
16	256	4096	0.427
64	256	16384	0.364
256	256	65536	0.330
1024	256	262144	0.320
4096	256	1048576	0.343
16384	256	4194304	0.448

Tabel 2: Uitvoeringstijd bij toenemend aantal blocks met striding.

Zoals te zien is in Tabel 2, neemt de tijd initieel significant af. Door het toevoegen van blocks worden meer Streaming Multiprocessors (SM's) op de GPU gelijktijdig ingezet, wat de parallellisatie ten goede komt.

Wat echter opvalt, is dat de rekentijd zijn minimum bereikt rond de 1024 blocks (0.320 ms) en daarna weer begint te stijgen bij 4096 blocks. Op een bepaald punt is er dus een saturatie.

Onze hardware heeft 384 CUDA cores ter beschikking, we zouden initieel dus kunnen verwachten dat we tussen 1 en 4 blocks satureren. Dit is echter niet het geval, omdat de GPU extra threads gereed



Figuur 2: Dalende trend in rekentijd met striding door het toevoegen van extra blocks. Het minimum is aangeduid

kan houden om verwerkt te worden zodra er hardwarecapaciteit vrijkomt, wat aanzienlijke tijdwinst oplevert. Daarnaast kan elke GPU core meerdere threads (/warps) runnen. Het is dus niet zo eenvoudig als zeggen dat het aantal "software"threads gelijk moet zijn aan het aantal cores. Ook zou het starten van overmatig veel blocks een negatief effect kunnen hebben omdat er meer overhead en scheduling complexiteit is, wat een toename in de tijd verklaart. Dit bewijst het nut van grid-striding.

4 Deel 2: Thread Divergence

In het tweede deel krijgen de rode pixels een 1D-Gaussian blur, de groene en blauwe pixels worden nog steeds enkel geïnverteerd. De GPU is ontworpen om alle 32 threads binnen een warp exact dezelfde instructie op exact hetzelfde moment te laten uitvoeren, hierdoor ontstaat er thread divergence wanneer sommige threads binnen een warp de filter moeten toepassen en andere threads de inversie. Een deel van de threads zal dus idle zijn en verspilt reken capaciteit tot de hele warp weer samen verder kan.

4.1 Divergence (Interleaved Format)

Standaard staat de data van onze afbeelding opgeslagen in het volgende patroon; RGBRGB.... Onze eerste kernel gebruikt een if/else structuur om te controleren of het een rode pixel (if) of een groene/blauwe pixel (else) is. Omdat **opeenvolgende pixels** door threads **in dezelfde warp** worden verwerkt, bevat elke warp threads die in de if-branch gaan en threads die in de else-branch gaan. De hardware kan deze twee verschillende operaties niet tegelijk doen. Ook hier is striding toegepast om meer flexibiliteit te hebben in het aantal blocks/threads dat we uitvoeren.

```
__global__ void inverseFilterRedStriding_Kernel(uint8_t* arrayOut, uint8_t* arrayIn,
    size_t arraySize)
{
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    for (int i = tid; i < arraySize; i += stride)
    {
        if(i % 3 == 0) arrayOut[i] = arrayIn[i-6]*0.1 + arrayIn[i-3]*0.25 + arrayIn[i
    ]*0.5 + arrayIn[i+3]*0.25 + arrayIn[i+6]*0.1;
        else arrayOut[i] = 255 - arrayIn[i];
    }
}
```

Blocks	Threads/block	Totale threads	Gemiddelde tijd (ms)
1	256	256	22.609
4	256	1024	3.323
16	256	4096	2.447
64	256	16384	2.222
256	256	65536	2.035
1024	256	262144	2.023
4096	256	1048576	2.015
16384	256	4194304	2.022

Tabel 3: Uitvoeringstijd divergente (interleaved) kernel met striding.

4.2 Non-Divergence (Planar Format)

Om dit op te lossen moeten we ervoor zorgen dat alle 32 threads in een warp hetzelfde doen. Dit doen we door de afbeeldingsdata vooraf te herformatteren naar een planair patroon, waarbij we alle kleuren groeperen in grote opeenvolgende blokken (RR...RGGG...GBB...B). De kernel bevat nog steeds dezelfde if/else logica, maar door de nieuwe data layout leest een warp nu bijvoorbeeld 32 opeenvolgende rode pixels. Hierdoor zal de volledige warp tegelijk de filter uitvoeren. Een andere warp krijgt later enkel groene of blauwe pixels en zal enkel de inversie operatie uitvoeren. De branch-divergence is zo opgelost. Vervolgens moesten we dit planaire formaat ook terug herformatteeren naar het standaard interleaved formaat zodat onze image viewer de foto van Anna correct terug kan uitlezen.

```
__global__ void inverseFilteredRedFormatStriding_Kernel(uint8_t* arrayOut, uint8_t*
    arrayIn, size_t arraySize)
{
    // 1. Calculate the starting thread index and the total grid stride
    int tid = threadIdx.x + blockIdx.x * blockDim.x;
    int stride = blockDim.x * gridDim.x;

    // 2. Loop over the array using the stride
    for (int i = tid; i < arraySize; i += stride)
    {
        if(i < arraySize / 3) arrayOut[i] = arrayIn[i-2]*0.1 + arrayIn[i-1]*0.25 +
    arrayIn[i]*0.5 + arrayIn[i+1]*0.25 + arrayIn[i+2]*0.1;
        else arrayOut[i] = 255 - arrayIn[i];
    }
}
```

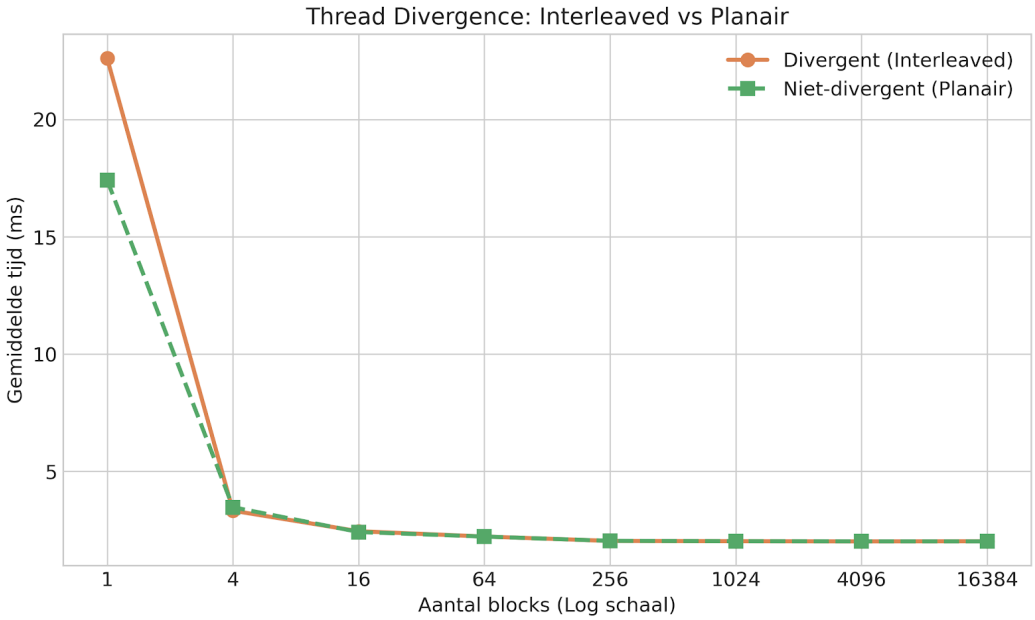
Blocks	Threads/block	Totale threads	Gemiddelde tijd (ms)
1	256	256	17.418
4	256	1024	3.471
16	256	4096	2.414
64	256	16384	2.220
256	256	65536	2.038
1024	256	262144	2.024
4096	256	1048576	2.018
16384	256	4194304	2.016

Tabel 4: Uitvoeringstijd niet-divergente (planaire) kernel met striding.

Wanneer we de tabellen vergelijken, valt op dat de planaire (non-divergente) kernel bij **1 block** sneller is (17.4 ms tegenover 22.6 ms voor de divergente kernel). Omdat er bij 1 block slechts een beperkt aantal warps actief is, weegt de rekentijd van de idle threads zwaarder door. Door de data planair te formatteren merken we hier de prestatiewinst.

Zodra we echter **4 of meer blocks** starten, zijn de resultaten gelijkaardig en is de divergente kernel zelfs een heel klein beetje sneller (bijv. 3.32 ms vs 3.47 ms bij 4 blocks). Voor dit kleine verschil zie ik drie mogelijke oorzaken (speculatief):

1. **Dynamic Frequency Scaling:** In tegenstelling tot deel1 was de GPU clockfrequentie hier niet vastgezet (geen sudo toegang tijdens deze labo sessie). Eventueel heeft de GPU door het verlies



Figuur 3: Vergelijking van divergentie (interleaved data) en non-divergentie (planaire data)

aan performantie, of een gelijkaardige reden, de frequentie hier hoger gezet. Helaas hebben we dit tijdens de metingen niet in het oog kunnen houden.

2. **Optimalisatie van de GPU:** Eventueel zijn er optimalisaties die de GPU uitvoert waarvan we niet op de hoogte zijn

3. **(Memory) Bottleneck:** Misschien is er een bottleneck die niets te maken heeft met de uitvoering van de thread operaties. Eventueel zou het voorzien van de data/pixels aan de threads bij een toename aan blocks een bottleneck kunnen worden.

5 Conclusie

In dit labo zien we dat efficientie bij GPU-programmering meer is dan gewoon het aantal threads verhogen, maar dat er bepaalde technieken zijn om slimmer om te gaan met de data en hardware die je hebt.

De **Stride looping** opdracht bewijst dat het starten van meer blocks/threads niet altijd resulteert in tijdwinst, eventueel door de extra scheduling overhead die hierbij komt kijken. Bovendien bevestigden de tests zonder striding dat blockgroottes die geen veelvoud zijn van 32 leiden tot inactieve threads en verspilde capaciteit binnen een warp.

De **Thread divergence** opdracht toont duidelijk aan dat de data structuur een grote impact kan hebben op thread diverence en dus de efficiëntie. Ook zagen we opnieuw een mogelijke bottleneck van de memory.